

Desiree - reset

Review of what went wrong, what is kept, and the design going forward. 13 September 2026.

1. Why this is being reset

Yesterday produced eleven scripts, five test harnesses and a working demo, and the demo still could not reliably answer *what is on my screen*. The individual bugs were real, but they were symptoms. Seven root causes, stated plainly because they are the things the new design has to prevent rather than patch.

Root cause	What it produced	What prevents it now
Code before design	Every turn shipped code; every patch created the next bug	This document precedes any code. Each step has a gate.
Routers instead of tool use	SCREEN/CHAT, then deixis, then MEMORY, then LOOKUP, then shape - every new question broke the last classifier	One agent loop. The model selects tools. No classifier anticipates question shapes.
Prompt rules used as architecture	'Never guess' broke general knowledge; 'cover every region' overrode the actual question	Grounding is a harness property carried by tool results, not a sentence in a system prompt
Assistant prose treated as record	Her descriptions in history were copied verbatim over a screen she could see - ADR-0002 at the prompt layer	Her output is never re-injected as evidence. Memory is data. History carries user turns and tool results.
Overfitting to the example in front of me	Prices became a catalogue; prices became a consensus engine; both failed on the next question	Tools return neutral structured data. Interpretation happens in the loop, per question.
Asserting instead of measuring	'OCR is ~100ms' (was 7s); 'num_ctx 2048 is fine' (evicted the VLM); 'thirds are windows'	Every claim about latency, memory or accuracy is a number from a run, stated with its conditions
Ground truth weaker than the system under test	Five automated scorers, each wrong in a different way	Gates are judged by you against what you can see. Automated scoring only where truth is exact (tool selection).

2. What is proven, and kept

These are measurements, not opinions. Everything else from yesterday is discarded or re-derived from these.

Finding	Measurement	Consequence
Co-residency	llama3.1:8b Q4 + qwen2.5vl:7b at 8k ctx: 13.8-14.1 GB peak of 15.9, 1.8-1.9 GB free. 16k context evicts.	This is the resident set. 8k is the hard ceiling. No second machine is justified.
Swap cost	Evict-and-reload thrash: 5.1 s median first audio	The chat model is pinned. Nothing else is ever allowed to evict it.
Vision latency	One 1280px tile: 0.45-0.7 s prefill. Two tiles: 1.35-1.6 s.	One tile. The VLM describes; it does not read small text.
OCR	Full 3440x1440 in 1.1-1.6 s with 3 parallel single-threaded tesseract processes; 2.2-7.1 s serial	Parallel by region, CPU only, zero VRAM. Always-on watching is affordable.
Speculative capture	Capture + OCR during speech: screen turn 1.3-1.5 s first audio vs 3.9 s serial	Kept. Re-verify at end of speech with a tolerant diff, not an exact hash.
VLM on values	Misread prices repeatedly (\$7777, \$9999, \$1500 for \$1155); honest about absence (80/80 absent-object probes clean)	OCR reads text and numbers. The VLM never supplies a number.
Voice	STT 0.02 s on GPU; chat first audio 0.13-0.3 s with streaming TTS	Proven since POC 1. Untouched.
ADR-0002 generalises	A past 'I don't know' in history beat the OCR in front of her	Applies to every path where her output returns as input, not just memory files

3. What is discarded

Component	Why it goes
All routers and classifiers (SCREEN_WORDS, DEIXIS, MEMORY_WORDS, FOLLOWUP, SUPERLATIVE, SHAPES)	Replaced by model tool selection. Each was a partial enumeration of question shapes.
prioritise() and every 'cover all regions' style prompt rule	Reordering buried the browser; the rule then overrode the question. Both are architecture done with prompts.
Vertical-strip regions	Thirds happened to match one layout. Windows are real; the OS reports them.
catalogue.py, observations.py	Overfit to prices and to time series respectively. recall.py's episode model is the general one.
Interest lifecycle (cold / ignored / retired)	Premature. The extraction that feeds it was filing bare nouns. Revisit when interests exist and misbehave.
The five scoring harnesses	Each had ground truth weaker than the model. Kept only as a record of what not to do.
research.py shape classifier	Same router mistake, one layer up. The loop decides when to research; research returns claims.

4. The architecture

There is no router. One loop, five tools, provenance on every result.

```
user turn (voice or text)
  → model receives: system, tool schemas, user turns so far, prior TOOL RESULTS (never prior
  assistant prose)
  → model emits zero or more tool calls
  → harness executes each (named, fixed schema, sandboxed), returns structured data +
  provenance
  → repeat, max 4 steps
  → model answers; harness streams to TTS
```

Why this is not a violation of ADR-0001. ADR-0001 forbids a model deciding whether an action is safe. Choosing which named tool answers a question is capability selection, not a safety gate. The safety property comes from every tool being named, fixed-argument and sandboxed - which is exactly what ADR-0001 asks for. Yesterday's routers were a misreading of it.

Why one model, chosen for tool calling. The loop driver must reliably select the right tool, fill valid arguments, and never invent a tool. That is a measured property - the Berkeley Function Calling Leaderboard measures precisely it - and small models vary widely. Step 0 below selects the driver by test, not preference. Candidates are all 8B Q4, so the resident set does not change.

4b. The second entry point: things she notices

The loop above is entered by a user turn. A person monitoring a screen has two triggers, not one: things they are asked, and things they notice. Same attention, different entry.

```
PERCEPTION  watcher - continuous, CPU only, deterministic; every settled screen becomes an
episode (logged, no model)
SALIENCE    cheap deterministic filter: matches a standing interest? tracked number crossed a
threshold?
ATTENTION   the SAME loop, entered with an OBSERVATION instead of a question; model decides
speak / silent / file
SOCIAL      never while the user is speaking; rate-limited; strong default to silence
```

Salience is deterministic and cheap by design. The model is not consulted about every screen change - that is both expensive and produces a nag. It sees only events that pass the filter, and its job is then a judgement: is this worth interrupting for. Perception is continuous; attention is rare. Two event classes to start: an interest match (the user said they wanted it) and a threshold crossing (arithmetic over a logged series). Novelty alone does not reach the model, because 'you opened a new tab' is never worth saying.

This is build step 4. It adds a second way into the loop and changes nothing before it.

5. Tools

Every tool is a thin wrapper over a module that is tested standalone before the loop exists. Results are JSON with a *source* field the model is required to carry into its answer.

Tool	Arguments	Returns	Backed by
read_screen	-	one line per open window: app, title, first content line	windows.py + screenread.py
read_window	app: str	full OCR text of that window, with label/value pairs where the layout has them	screenread.py
search_memory	cue: str	episodes matching the cue: when, how long, entities, confidence	recall.py
research	question: str	verified claims with quotes and domains; agreement counts; NOT FOUND if none verify	research.py (map-reduce, quote-verified)
remember	want: str	acknowledgement; filed as a standing interest	recall.py

Deliberately absent: a shell, a file tool, an eval. Deliberately absent for now: describe_visually (the VLM). It is added as a sixth tool after the loop is proven on five, because it is the one that can hallucinate and it should arrive with its own gate.

6. Grounding, as a harness property

The prompt does not say 'never guess'. Instead: every tool result carries a source, the system prompt says answers about the screen, memory or the web must cite the tool that produced them, and anything else is prefixed *from what I know*. That is one rule, and it is enforceable: the harness can check that a screen answer followed a screen tool call, and log when it did not. Yesterday's failures - the fabricated integration, the 1982 Philippines date, the CPU at \$13,499 - were all answers with no tool result behind them.

7. Memory, as data

Episodes (recall.py) stay: timestamped, merged, entity-extracted, searchable by cue, with confidence. Her prose is never stored and never re-injected. Conversation history sent to the model is *user turns and tool results only*. This is the single largest fix and it costs nothing.

8. Build order, with gates

Nothing is built until the previous gate is confirmed by you, on your machine, against what you can see. This is the mechanism that replaces 'no mistakes' as a promise with 'no unverified step' as a process.

Step	Deliverable	Gate (you confirm)
0	tool_smoke.py - 20 scripted questions, three candidate driver models, exact-match scoring on tool selection	One model scores $\geq 18/20$ with zero invented tools. That model becomes the driver.
1	windows.py + screenread.py validated standalone on three layouts (tiled, one maximised, overlapping)	App labels correct; per-window text readable; spec pairs extracted from the PLE page
2	agent.py - the loop, text in / text out, five tools, no voice, no watcher	10 scripted questions answered correctly by eye, including the two-step 'most expensive part' case
3	Voice wired on (POC 1/4 code, unchanged)	Chat first audio < 0.5 s; screen first audio < 2.5 s; both models resident throughout
4	Watcher on (settle-based, CPU only, exclusions verified)	'frames excluded' moves when a matching window is focused; OCR passes stay low
5	describe_visually tool added	Absent-object probes clean; she flags visual answers as impressions
6	remember + interest alerts	One stated want fires on a real match; one passing mention does NOT get filed

Each step is one file or one wiring change. If a gate fails, the step is fixed before anything else is touched - and fixing means changing the design if the design is wrong, not adding a rule to a prompt.

9. What 'no mistakes' actually means here

I cannot run your GPU, your X server or tesseract from here, so I cannot promise code that is bug-free on first run - and yesterday's damage came from pretending otherwise. What I can do is keep every step small enough to verify in minutes, state what each one assumes, and not build on a step you have not confirmed. The mistakes yesterday were not typos; they were building three floors on an unchecked foundation. The gates are the fix for that.

10. Step 0, now

tool_smoke.py stubs the five tools, sends twenty questions to each candidate through Ollama's tools API and scores tool choice, argument validity, invented tools and the two-step chain. **Result, 13 September:** llama3.1:8b 16/20; qwen3:8b 19/20, zero invented - clears the gate and is the driver; llama3-groq-tool-use:8b 9/20 with eleven no-calls through Ollama's generic template. qwen3's one miss is the two-step case (answered from knowledge without looking) - noted for step 2, not disqualifying.